

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)
Applied Business Analytics Practical

Practical – X

Roll No.: T001	Name: Soham Acharekar
Class: TYIT	Batch: 01
Date of Assignment: 10/3/2026	Date/Time of Submission: 10/3/2026

Practical 10: Regression Analysis and Time Series Analysis

Part A: Regression Analysis

Dataset: sales_data.csv

```
!pip install pandas numpy matplotlib scikit-learn statsmodels
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_squared_error
from statsmodels.tsa.holtwinters import ExponentialSmoothing, SimpleExpSmoothing
from statsmodels.tsa.arima.model import ARIMA
```

- 1. Load the sales_data.csv dataset.**
- 2. Display the first five records.**
- 3. Identify the independent and dependent variables.**
- 4. Select advertising as the independent variable and sales as the dependent variable.**
- 5. Check the data for missing values.**

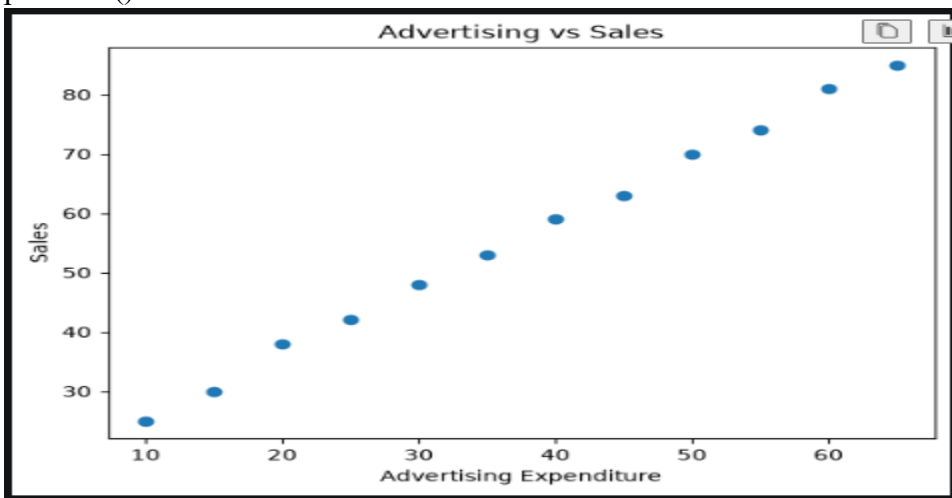
```
# Load the dataset
df = pd.read_csv("sales_data.csv")
# Display the first five records
print(df.head())
# Display column names
print(df.columns)
# Identify independent and dependent variables
X = df[["advertising"]]
y = df["sales"]
print("Independent Variable: Advertising")
print("Dependent Variable: Sales")
# Check missing values
print(df.isnull().sum())
```

```
df = df.dropna(subset=["advertising", "sales"])
X = df[["advertising"]]
y = df["sales"]
```

```
...      advertising  sales
0         10      25
1         15      30
2         20      38
3         25      42
4         30      48
Index(['advertising', 'sales'], dtype='str')
Independent Variable: Advertising
Dependent Variable: Sales
advertising    0
sales          0
dtype: int64
```

6. Create a scatter plot of advertising expenditure and sales.

```
plt.scatter(df["advertising"], df["sales"])
plt.xlabel("Advertising Expenditure")
plt.ylabel("Sales")
plt.title("Advertising vs Sales")
plt.show()
```



7. Develop a simple linear regression model.

8. Display the regression equation.

```
# Create and train the model
model = LinearRegression()
model.fit(X, y)
# Display the regression equation
b0 = model.intercept_
b1 = model.coef_[0]
print("Intercept:", b0)
print("Slope:", b1)
print("Regression Equation: Sales =", b0, "+", b1, "* Advertising")
```

```
Intercept: 14.705128205128226
Slope: 1.0923076923076918
Regression Equation: Sales = 14.705128205128226 + 1.0923076923076918 * Advertising
```

9. Use the model to predict sales for a given advertising expenditure.

```
advertising_value = pd.DataFrame({"advertising": [50]})
predicted_sales = model.predict(advertising_value)
print("Predicted Sales:", predicted_sales[0])
```

```
Predicted Sales: 69.32051282051282
```

10. Calculate the R2 score of the model.**11. Calculate the Mean Squared Error (MSE).**

```
# Predict sales for all records
y_pred = model.predict(X)
# Calculate R2 score
r2 = r2_score(y, y_pred)
# Calculate Mean Squared Error
mse = mean_squared_error(y, y_pred)
print("R2 Score:", r2)
print("Mean Squared Error:", mse)
```

```
R2 Score: 0.998313669479218
Mean Squared Error: 0.6004273504273502
```

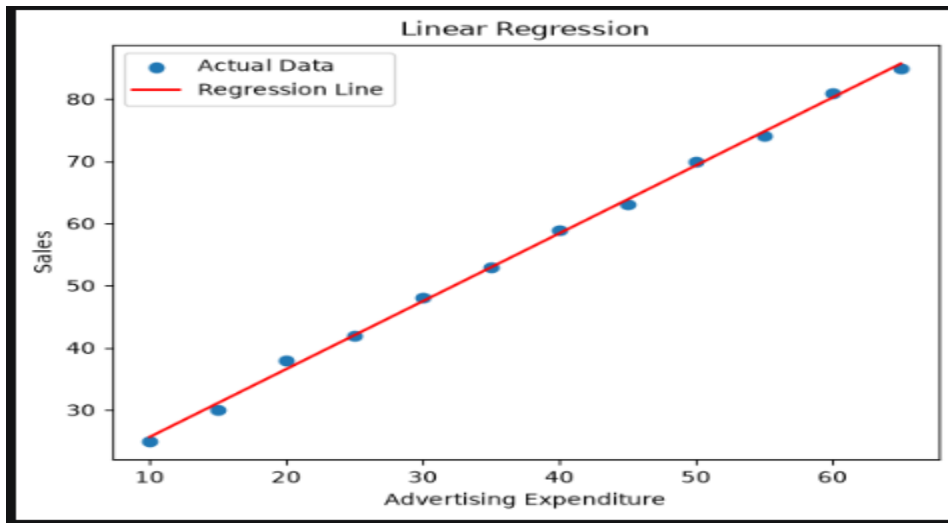
12. Compare the actual sales with the predicted sales.

```
comparison = pd.DataFrame({
    "Actual Sales": y,
    "Predicted Sales": y_pred
})
print(comparison)
```

	Actual Sales	Predicted Sales
0	25	25.628205
1	30	31.089744
2	38	36.551282
3	42	42.012821
4	48	47.474359
5	53	52.935897
6	59	58.397436
7	63	63.858974
8	70	69.320513
9	74	74.782051
10	81	80.243590
11	85	85.705128

13. Plot the regression line with the actual data points.

```
plt.scatter(X["advertising"], y, label="Actual Data")
plt.plot(X["advertising"], y_pred, color="red",
        label="Regression Line")
plt.xlabel("Advertising Expenditure")
plt.ylabel("Sales")
plt.title("Linear Regression")
plt.legend()
plt.show()
```



14. Interpret the regression results.

Interpretation:

The regression model studies the relationship between advertising expenditure and sales.

A positive slope indicates that sales tend to increase as advertising expenditure increases.

A negative slope indicates that sales tend to decrease as advertising expenditure increases.

The R^2 score indicates how well the model explains sales variation.

The MSE measures the prediction error.

15. Write a conclusion about how advertising expenditure is related to sales.

A simple regression model is:

$$\text{Sales} = b_0 + b_1(\text{Advertising})$$

Conclusion: Simple linear regression is used to analyse the relationship between advertising expenditure and sales. The model predicts sales based on advertising expenditure. The slope, R^2 score, and MSE help interpret the relationship and evaluate the model's performance. The actual conclusion should be based on the calculated results.

Part B: Time Series Analysis

Dataset: monthly_sales.csv

The dataset can contain:

Month Sales

Jan 120

Feb 135

Mar 128

Apr 145

May 150

Jun 160

Jul 155

Aug 170

Sep 165

Oct 180

Nov 190

Dec 200

Steps:**16. Load monthly sales data.**

```
ts_df = pd.read_csv("monthly_sales.csv")
print(ts_df.head())
```

	Month	Sales
0	Jan	120
1	Feb	135
2	Mar	128
3	Apr	145
4	May	150

```
ts_df = pd.DataFrame({
    "Month": ["Jan", "Feb", "Mar", "Apr", "May", "Jun",
             "Jul", "Aug", "Sep", "Oct", "Nov", "Dec"],
    "Sales": [120, 135, 128, 145, 150, 160,
             155, 170, 165, 180, 190, 200]
})
print(ts_df)
```

	Month	Sales
0	Jan	120
1	Feb	135
2	Mar	128
3	Apr	145
4	May	150
5	Jun	160
6	Jul	155
7	Aug	170
8	Sep	165
9	Oct	180
10	Nov	190
11	Dec	200

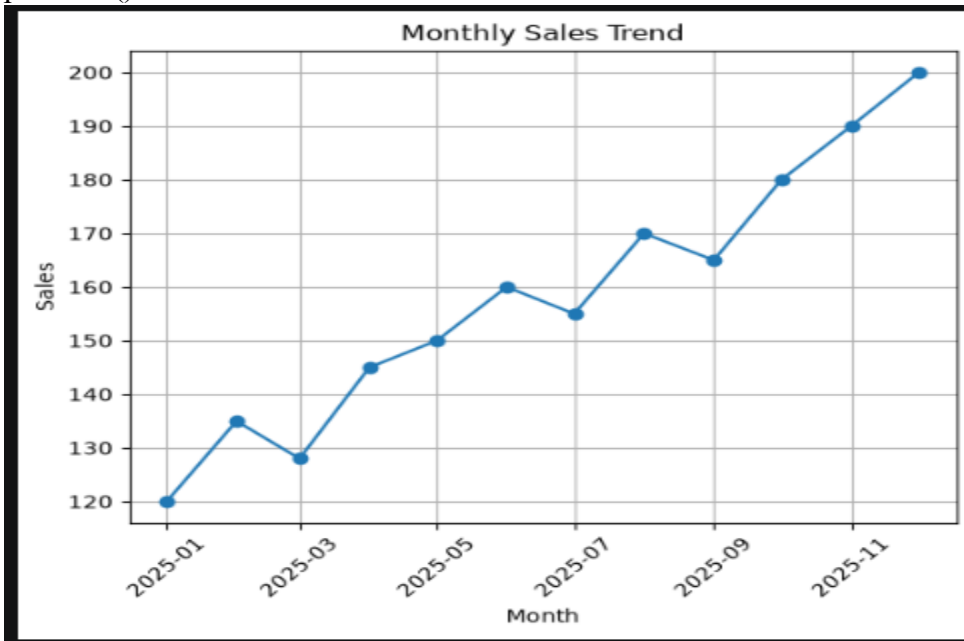
17. Convert the date column into date format.

```
ts_df["Date"] = pd.date_range(
    start="2025-01-01",
    periods=len(ts_df),
    freq="MS"
)
ts_df = ts_df.set_index("Date")
print(ts_df)
```

Date	Month	Sales
2025-01-01	Jan	120
2025-02-01	Feb	135
2025-03-01	Mar	128
2025-04-01	Apr	145
2025-05-01	May	150
2025-06-01	Jun	160
2025-07-01	Jul	155
2025-08-01	Aug	170
2025-09-01	Sep	165
2025-10-01	Oct	180
2025-11-01	Nov	190
2025-12-01	Dec	200

18. Plot monthly sales using a line chart.**19. Observe the trend.**

```
plt.plot(ts_df.index, ts_df["Sales"], marker="o")
plt.xlabel("Month")
plt.ylabel("Sales")
plt.title("Monthly Sales Trend")
plt.xticks(rotation=45)
plt.grid(True)
plt.show()
```



Observation: The sample sales data shows an overall upward trend from January to December, although sales decrease slightly in some months.

20. Apply Exponential Smoothing.

```
es_model = SimpleExpSmoothing(
    ts_df["Sales"]
).fit(optimized=True)
es_forecast = es_model.forecast(3)
print("Exponential Smoothing Forecast:")
print(es_forecast)
```

```
Exponential Smoothing Forecast:
2026-01-01    199.60495
2026-02-01    199.60495
2026-03-01    199.60495
Freq: MS, dtype: float64
c:\Users\IT-32\AppData\Local\Programs\Python\Python313\Lib\site-packages\statsmodels
self._init_dates(dates, freq)
```

21. Forecast the next 3 months.

```
print("Next 3 Months Forecast:")
print(es_forecast)
```

```

Next 3 Months Forecast:
2026-01-01    199.60495
2026-02-01    199.60495
2026-03-01    199.60495
Freq: MS, dtype: float64

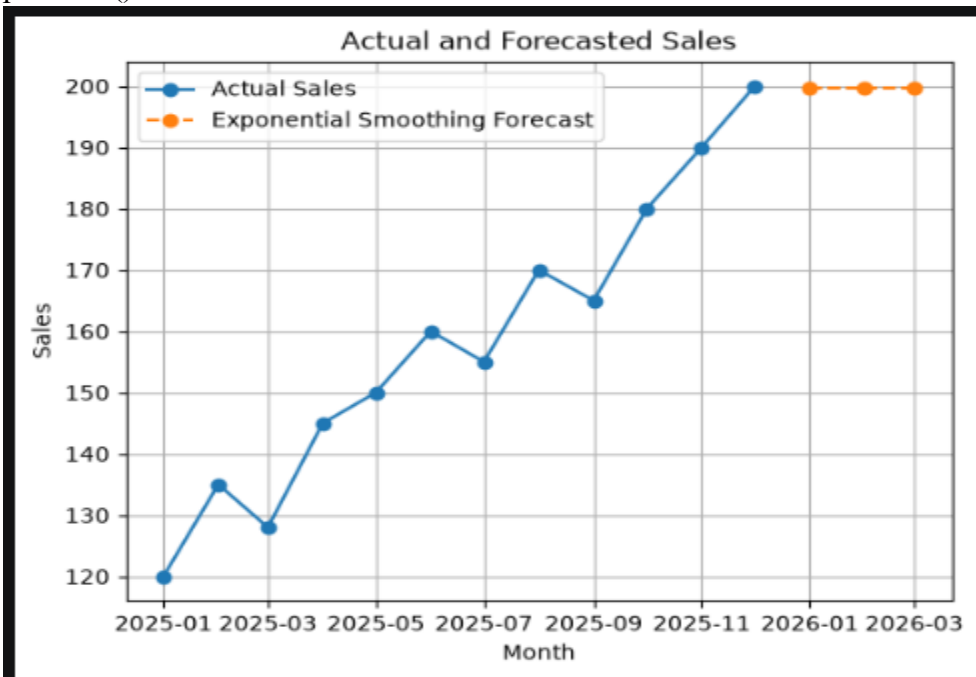
```

22. Plot actual and forecasted sales.

```

future_dates = pd.date_range(
    start=ts_df.index[-1] + pd.offsets.MonthBegin(1),
    periods=3,
    freq="MS"
)
plt.plot(ts_df.index, ts_df["Sales"],
         marker="o", label="Actual Sales")
plt.plot(future_dates, es_forecast.values,
         marker="o", linestyle="--",
         label="Exponential Smoothing Forecast")
plt.xlabel("Month")
plt.ylabel("Sales")
plt.title("Actual and Forecasted Sales")
plt.legend()
plt.grid(True)
plt.show()

```



23. Calculate forecasting error.

```

train = ts_df["Sales"].iloc[:-3]
test = ts_df["Sales"].iloc[-3:]

```

```

es_test_model = SimpleExpSmoothing(
    train
).fit(optimized=True)
es_test_forecast = es_test_model.forecast(3)
es_mse = mean_squared_error(test, es_test_forecast)
print("Exponential Smoothing MSE:", es_mse)

```

```

Exponential Smoothing MSE: 670.8826344294527
c:\Users\IT-32\AppData\Local\Programs\Python\Python313\Lib\site-packages\statsmodels
self._init_dates(dates, freq)

```

24. Apply ARIMA to the same data.

```

arima_model = ARIMA(
    ts_df["Sales"],
    order=(1, 1, 1)
).fit()
print(arima_model.summary())

```

```

SARIMAX Results
=====
Dep. Variable:      Sales      No. Observations:      12
Model:              ARIMA(1, 1, 1)      Log Likelihood      -40.405
Date:               Sat, 03 Oct 2026      AIC      86.811
Time:               10:28:17      BIC      88.005
Sample:             01-01-2025      HQIC      86.058
                  - 12-01-2025
Covariance Type:    opg
=====
              coef      std err          z      P>|z|      [0.025      0.975]
-----
ar.L1          -1.0000         0.003    -296.271      0.000      -1.007      -0.993
ma.L1           0.9960         0.576       1.730      0.084      -0.133       2.125
sigma2          74.9840         0.008    9519.129      0.000      74.969      74.999
=====
Ljung-Box (L1) (Q):              1.19      Jarque-Bera (JB):              0.57
Prob(Q):                        0.27      Prob(JB):              0.75
Heteroskedasticity (H):          0.98      Skew:              0.40
Prob(H) (two-sided):            0.98      Kurtosis:             2.21
=====

Warnings:
[1] Covariance matrix calculated using the outer product of gradients (complex-step).
[2] Covariance matrix is singular or near-singular, with condition number 2.11e+20. St
c:\Users\IT-32\AppData\Local\Programs\Python\Python313\Lib\site-packages\statsmodels\t
self._init_dates(dates, freq)
c:\Users\IT-32\AppData\Local\Programs\Python\Python313\Lib\site-packages\statsmodels\t
self._init_dates(dates, freq)
c:\Users\IT-32\AppData\Local\Programs\Python\Python313\Lib\site-packages\statsmodels\t
self._init_dates(dates, freq)
c:\Users\IT-32\AppData\Local\Programs\Python\Python313\Lib\site-packages\statsmodels\b
warnings.warn("Maximum Likelihood optimization failed to "

```

25. Forecast the next 3 months using ARIMA.

```

arima_forecast = arima_model.forecast(steps=3)
print("ARIMA Forecast:")
print(arima_forecast)

```

```

ARIMA Forecast:
2026-01-01    193.259353
2026-02-01    199.999922
2026-03-01    193.259431
Freq: MS, Name: predicted_mean, dtype: float64

```

26. Compare the ARIMA and Exponential Smoothing forecasts.

```
forecast_comparison = pd.DataFrame({
    "Exponential Smoothing": es_forecast.values,
    "ARIMA": arima_forecast.values
}, index=future_dates)
print(forecast_comparison)
```

	Exponential Smoothing	ARIMA
2026-01-01	199.60495	193.259353
2026-02-01	199.60495	199.999922
2026-03-01	199.60495	193.259431

27. Calculate and compare their forecasting errors.

ARIMA trained on the same training period

```
arima_test_model = ARIMA(
    train,
    order=(1, 1, 1)
).fit()
arima_test_forecast = arima_test_model.forecast(steps=3)
# Calculate test errors
arima_mse = mean_squared_error(test, arima_test_forecast)
print("Exponential Smoothing MSE:", es_mse)
print("ARIMA MSE:", arima_mse)
```

```
Exponential Smoothing MSE: 670.8826344294527
ARIMA MSE: 470.80210263438283
c:\Users\IT-32\AppData\Local\Programs\Python\Python313\Lib\site-packages\statsmodels\
self._init_dates(dates, freq)
c:\Users\IT-32\AppData\Local\Programs\Python\Python313\Lib\site-packages\statsmodels\
self._init_dates(dates, freq)
c:\Users\IT-32\AppData\Local\Programs\Python\Python313\Lib\site-packages\statsmodels\
self._init_dates(dates, freq)
c:\Users\IT-32\AppData\Local\Programs\Python\Python313\Lib\site-packages\statsmodels\
warnings.warn("Maximum Likelihood optimization failed to "
```

28. Identify which model gives the lower forecasting error for the given dataset.

```
if es_mse < arima_mse:
    print("Exponential Smoothing has lower forecasting error.")
elif arima_mse < es_mse:
    print("ARIMA has lower forecasting error.")
else:
    print("Both models have equal forecasting error.")
```

```
ARIMA has lower forecasting error.
```

29. Write the final conclusion.

Conclusion: Time Series Analysis is used to study sales over time and forecast future sales. A line chart helps identify the overall trend. Exponential Smoothing and ARIMA are used to forecast the next three months. Their forecasting errors are calculated on the same test period and compared. The model with the lower test MSE has smaller squared prediction errors for that period.